Large Language Models (LLM) and Temporal Logic of Actions (TLA): How Effective are LLMs for Verification Systems

Brian Ortiz, Arslan Bisharat, Eric Spencer, Khushboo Bhadauria, Mohammed Abuhamad, Konstantin Laüfer, George K. Thiruvathukal, Taining Wang

INTRODUCTION

Abstract

- TLA+ is formal specification language used to model, design, verify, and debug complex concurrent and distributed systems. TLA+ supports formal verification of software at companies such as Amazon and Microsoft and Amazon
- Formal Methods like TLA+ are powerful tools to promote software reliability, but are difficult to learn and apply
- LLMs offer an opportunity to automate formal model synthesis
- Our work presents a systematic evaluation of LLM-based TLA+ specification synthesis from natural language

Gap in Prior Work

- Existing GenAI research on TLA+ focuses on:
  - Creation of specifications from code
  - Enforcement of syntax through token constraints
  - Use of specifications to guide code generation
- No systematic study has tested whether:
  - LLMs can produce full and semantically correct TLA+ specifications directly from natural language
  - What error patterns emerge across model architectures
  - How prompting strategies affect generation quality

Research Questions

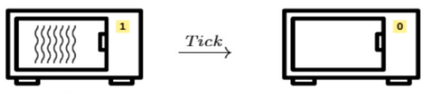
RQ1: Can LLMs generate TLA+ specifications that are both syntactically correct and semantically faithful?
RQ2: What error types do different LLM backends produce?
RQ3: How do quality metrics correlate with model architecture, size and fine-tuning?
RQ4: What failure modes and hallucinations recur in LLM-generated specifications?

Contributions

- We curate a dataset for natural-language-to-TLA+ specification synthesis, consisting of 205 TLA+ specifications paired with natural language comments and TLC configurations
- We provide a systematic evaluation of LLM-based TLA+ specification generation/synthesis from natural language, testing across 30 LLMs from eight families across four prompting strategies
- We establish quantified baselines for TLA+ synthesis and highlight the gap between current LLM capabilities and reliable specification synthesis

Background

- Developed by Leslie Lamport in 1999
- TLA+ (Temporal Logic of Actions) was created to describe concurrent and distributed systems behavior through mathematical formulas
- Combines temporal logic, first-order logic, and set theory



$$
\begin{aligned}
Tick \triangleq\ & \wedge radiation = ON \\
& \wedge timeRemaining' = timeRemaining - 1 \\
& \wedge timeRemaining' \geq 0 \\
& \wedge \text{IF } timeRemaining' = 0 \\
& \quad \text{THEN } radiation' = OFF \\
& \quad \text{ELSE UNCHANGED } \langle radiation \rangle \\
& \wedge \text{UNCHANGED } \langle door \rangle
\end{aligned}
$$

Figure 3. *Tick* action for decrementing the timer, defined as a conjunction of preconditions, effects, and postconditions, where v' refers to a variable's value in the next state following the action. According to the preconditions, this action is enabled only when the oven is radiating and there is nonzero time remaining. Its effect is to decrement the remaining time; when the remaining time reaches zero, its additional effect is to shut off radiation.

Example: TLA+ syntax modeling microwave *Tick* action

RESULTS

RQ1 Summary
LLMs produce syntactically valid TLA+ in up to 26.6% of attempts, but semantic correctness is rare (8.6% TLC pass rate using Progressive Prompting strategy). Only Progressive prompting achieved any TLC passes, led by DeepSeek R1:8B (14/26) and Qwen3 models (10-12 TLC passes each). Completion strategies show low textual similarity (BLEU < 0.08 and ROUGE-L < 0.22).

RQ2 Summary
"Parse: Bad Module Body" accounts for roughly half of all failures across every strategy and points to a structural failure mode that future datasets should target. Full generation strategies fail early with specific illegal tokens; completion strategies fail mid-to-late with structural mismatches. Lexical contamination from non-TLA+ languages (Unicode operators, back-ticks, semicolons) is systematic and strategy dependent. Grammar-constrained generation and deterministic post-processing offer concrete paths for future LLM improvement.

RQ3 Summary
Standard assumptions about scale and specialization do not hold for TLA+. Smaller reasoning models (DeepSeek R1:8B, Llama 3.2:1B) outperform large siblings. Code-specialist models consistently underperform general-purpose ones due to negative transfer from mainstream language training. Model family determines strategy fit better than overall capability.

RQ4 Summary
Five systematic hallucination categories recur across all models (1) Unicode operator substitution, (2) cross-language syntax injection (semicolons in 12.6% of files, backticks in 5.9%), (3) reasoning token leakage (<think> blocks in Qwen Q), (4) generation length miscalibration (0.00-9.84x ground truth), and (5) structural errors (234+ missing ==== terminators). Each category is traceable to specific training data bias and addressable by better datasets, grammar-specific decoding/post-processing.

Table 23: TLC model checking performance for Proprietary and Open-Source LLMs ($k = 3$). This table reports the binary success (PASS/FAIL) of 26 models across 26 TLA+ specifications, where a PASS indicates the specification successfully ran through the TLC model checker without errors.

Specification	claude-haiku-4-5	claude-opus-4-1	claude-sonnet-4-5	codellama:latest	deepseek-coder:6.7b	deepseek-coder:latest	deepseek-r1:1.5b	deepseek-r1:14b	deepseek-r1:32b	deepseek-r1:70b	deepseek-r1:7b	deepseek-r1:8b	gpt-oss:120b	granite3.3:latest	llama3.1:8b	llama3.2:1b	llama3.3:70b	llama3:latest	mistral:latest	phi4-mini:latest	qwen3:4b-thinking	qwq:latest	starling-lm:latest	gpt-4o	gpt-5	Total	Rate%
Alternate	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	0	0.0
BinarySearch	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✓	✗	✗	✗	✗	✗	1	3.8
BufferedRandomAccessFile	✗	✗	✓	✗	✗	✗	✗	✗	✓	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	2	7.7
ClientCentric	✗	✗	✓	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	1	3.8
Consensus	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✓	✗	✗	✗	✗	✗	1	3.8
EWD687a_anim	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✓	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	1	3.8
EWD998ChanID_export	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✓	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	1	3.8
EnvironmentController	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✓	✗	✗	✗	✗	✓	2	7.7
HDiskSynod	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✓	1	3.8
Hanoi	✗	✗	✓	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	1	3.8
KVsnap	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✓	1	3.8
MCConsensus	✗	✗	✓	✗	✗	✗	✗	✗	✓	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✓	4	15.4
MCDieHarder	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	0	0.0
MCEWD687a	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	0	0.0
Majority	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✓	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	1	3.8
MajorityProof	✓	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	2	7.7
MultiPaxos	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	0	0.0
MultiPaxos_MC	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	0	0.0
Relation	✓	✓	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	3	11.5
SimpleRegular	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✓	✗	✗	✗	1	3.8
TLAPS	✓	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✓	2	7.7
TwoPhase	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✓	✓	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	3	11.5
VoucherCancel	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	1	3.8
VoucherTransfer	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✓	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	1	3.8
YoYoNoPruning	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✓	✗	✗	✗	✗	1	3.8
nbacc_ray97	✗	✓	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✓	3	11.5
Total	3	1	3	3	0	0	0	0	1	2	0	6	2	0	0	0	0	0	0	2	3	0	0	1	7		
Rate %	**11.5**	**3.8**	**11.5**	**11.5**	**0.0**	**0.0**	**0.0**	**0.0**	**3.8**	**7.7**	**0.0**	**23.1**	**7.7**	**0.0**	**0.0**	**0.0**	**0.0**	**0.0**	**0.0**	**7.7**	**11.5**	**0.0**	**3.8**	**26.9**			

Table 22: SANY validation performance for Proprietary and Open-Source LLMs ($k = 3$). This table reports the binary success (PASS/FAIL) of 26 Large Language Models across 26 TLA+ specifications, including aggregate success rates and total pass counts per model.

Specification	claude-haiku-4-5	claude-opus-4-1	claude-sonnet-4-5	codellama:latest	deepseek-coder:6.7b	deepseek-coder:latest	deepseek-r1:1.5b	deepseek-r1:14b	deepseek-r1:32b	deepseek-r1:70b	deepseek-r1:7b	deepseek-r1:8b	gpt-oss:120b	granite3.3:latest	llama3.1:8b	llama3.2:1b	llama3.3:70b	llama3:latest	mistral:latest	phi4-mini:latest	qwen3:4b-thinking	qwq:latest	starling-lm:latest	gpt-4o	gpt-5	Total	Rate%
Alternate	✓	✓	✓	✗	✗	✗	✗	✗	✗	✗	✓	✗	✗	✓	✗	✗	✗	✗	✓	✓	✗	✗	✓	✓	✓	11	42.3
BinarySearch	✓	✓	✓	✓	✓	✗	✗	✗	✗	✗	✓	✗	✗	✓	✗	✗	✗	✗	✓	✓	✗	✗	✓	✓	✓	5	19.2
BufferedRandomAccessFile	✓	✓	✓	✓	✓	✗	✗	✗	✗	✗	✓	✗	✗	✓	✗	✗	✗	✗	✓	✓	✗	✗	✓	✓	✓	8	30.8
ClientCentric	✗	✓	✗	✗	✗	✗	✗	✗	✗	✗	✓	✗	✗	✓	✗	✗	✗	✗	✓	✓	✗	✗	✓	✓	✓	4	15.4
Consensus	✗	✓	✗	✗	✗	✗	✗	✗	✗	✗	✓	✗	✗	✓	✗	✗	✗	✗	✓	✓	✗	✗	✓	✓	✓	10	38.5
EWD687a_anim	✗	✓	✓	✗	✗	✗	✗	✗	✗	✗	✓	✗	✗	✓	✗	✗	✗	✗	✓	✓	✗	✗	✓	✓	✓	5	19.2
EWD998ChanID_export	✓	✓	✓	✗	✗	✗	✗	✗	✗	✗	✓	✗	✗	✓	✗	✗	✗	✗	✓	✓	✗	✗	✓	✓	✓	6	23.1
EnvironmentController	✗	✓	✓	✗	✗	✗	✗	✗	✗	✗	✓	✗	✗	✓	✗	✗	✗	✗	✓	✓	✗	✗	✓	✓	✓	5	19.2
HDiskSynod	✓	✓	✓	✗	✗	✗	✗	✗	✗	✗	✓	✗	✗	✓	✗	✗	✗	✗	✓	✓	✗	✗	✓	✓	✓	11	42.3
Hanoi	✓	✓	✓	✗	✗	✗	✗	✗	✗	✗	✓	✗	✗	✓	✗	✗	✗	✗	✓	✓	✗	✗	✓	✓	✓	10	38.5
KVsnap	✓	✓	✓	✗	✗	✗	✗	✗	✗	✗	✓	✗	✗	✓	✗	✗	✗	✗	✓	✓	✗	✗	✓	✓	✓	9	34.6
MCConsensus	✓	✓	✓	✗	✓	✓	✗	✓	✗	✓	✓	✗	✗	✓	✗	✗	✗	✗	✓	✓	✗	✗	✓	✓	✓	14	53.8
MCDieHarder	✓	✓	✓	✗	✗	✗	✗	✗	✗	✗	✓	✗	✗	✓	✗	✗	✗	✗	✓	✓	✗	✗	✓	✓	✓	8	30.8
MCEWD687a	✓	✓	✓	✗	✗	✗	✗	✗	✗	✗	✓	✗	✗	✓	✗	✗	✗	✗	✓	✓	✗	✗	✓	✓	✓	12	46.2
Majority	✓	✓	✓	✗	✗	✗	✗	✗	✗	✗	✓	✗	✗	✓	✗	✗	✗	✗	✓	✓	✗	✗	✓	✓	✓	9	34.6
MajorityProof	✓	✓	✓	✗	✗	✗	✗	✗	✗	✗	✓	✗	✗	✓	✗	✗	✗	✗	✓	✓	✗	✗	✓	✓	✓	15	57.7
MultiPaxos	✓	✓	✓	✗	✗	✗	✗	✗	✗	✗	✓	✗	✗	✓	✗	✗	✗	✗	✓	✓	✗	✗	✓	✓	✓	9	34.6
MultiPaxos_MC	✓	✓	✓	✗	✗	✗	✗	✗	✗	✗	✓	✗	✗	✓	✗	✗	✗	✗	✓	✓	✗	✗	✓	✓	✓	9	34.6
Relation	✓	✓	✓	✗	✗	✗	✗	✗	✗	✗	✓	✗	✗	✓	✗	✗	✗	✗	✓	✓	✗	✗	✓	✓	✓	8	30.8
SimpleRegular	✓	✓	✓	✗	✗	✗	✗	✗	✗	✗	✓	✗	✗	✓	✗	✗	✗	✗	✓	✓	✗	✗	✓	✓	✓	7	26.9
TLAPS	✓	✗	✓	✗	✗	✗	✗	✗	✗	✗	✓	✗	✗	✓	✗	✗	✗	✗	✓	✓	✗	✗	✓	✓	✓	6	23.1
TwoPhase	✓	✓	✓	✗	✗	✗	✗	✗	✗	✗	✓	✗	✗	✓	✗	✗	✗	✗	✓	✓	✗	✗	✓	✓	✓	11	42.3
VoucherCancel	✓	✓	✓	✗	✗	✗	✗	✗	✗	✗	✓	✗	✗	✓	✗	✗	✗	✗	✓	✓	✗	✗	✓	✓	✓	6	23.1
VoucherTransfer	✓	✓	✓	✗	✗	✗	✗	✗	✗	✗	✓	✗	✗	✓	✗	✗	✗	✗	✓	✓	✗	✗	✓	✓	✓	10	38.5
YoYoNoPruning	✓	✓	✓	✗	✗	✗	✗	✗	✗	✗	✓	✗	✗	✓	✗	✗	✗	✗	✓	✓	✗	✗	✓	✓	✓	7	26.9
nbacc_ray97	✗	✓	✓	✗	✗	✗	✗	✗	✗	✗	✓	✗	✗	✓	✗	✗	✗	✗	✓	✓	✗	✗	✓	✓	✓	5	19.2
Total	21	23	24	6	3	1	2	3	7	4	0	6	18	4	3	3	13	13	6	2	2	6	3	1	20	26	215
Rate %	**80.8**	**88.5**	**92.3**	**23.1**	**11.5**	**3.8**	**7.7**	**11.5**	**26.9**	**15.4**	**0.0**	**23.1**	**69.2**	**15.4**	**11.5**	**11.5**	**50.0**	**50.0**	**23.1**	**7.7**	**7.7**	**23.1**	**11.5**	**3.8**	**76.9**	**100.0**		**31.8**

Table 11: Hallucination taxonomy across prompting strategies (RQ4). Integer values = file counts; ✓ = observed but count not isolated; 0 = not observed; n/a = not applicable. †FIM applies `clean_llm_response()` to strip markdown before validation. ‡File scan (contains ≥1 semicolon). §SANY first-error count (FS: 18/477; HC: 11/570; Prog. : 25/488; FIM: 10/558). ¶File scan shows higher rates (FS: 49/650 = 7.5%; HC: 23/650 = 3.5%; Prog. : 82/650 = 12.6%; FIM: 22/650 = 3.4%).

Pattern	Source	Correct TLA+	FS	HC	Prog	FIM
Unicode Operator Substitution (Lexical Error)						
∨ (U+2228)	LaTeX/Math	\/	✓	✓	✓	✓
∧ (U+2227)	LaTeX/Math	/\	✓	✓	✓	✓
∈ (U+2208)	LaTeX/Math	\in	✓	✓	✓	✓
→ (U+2192)	LaTeX/Math	->	✓	✓	✓	✓
Cross-Language Syntax Injection						
Semicolons (;)	C/Java/Py	*invalid*	18§/49¶	11§/23¶	25§/82¶	10§/22¶
Backticks (‘)	Markdown	*invalid*	28	10	21	13
END keyword	Pascal/SQL	====	0	0	✓	0
Reasoning/Formatting Leakage						
<think> blocks	CoT reasoning	*invalid*	15	30	n/a	28
Markdown fences	Output format	*invalid*	0	0	✓	0†
NL prose in body	Task confusion	*invalid*	0	0	✓	0
Structural Errors						
Missing ====	Incomplete gen	====	✓	✓	243+	✓
Missing MODULE	No preamble	- MODULE X -	0	0	✓	0
Duplicate MODULE	Repeated start	*single header*	0	0	✓	0

Table 9: SANY error taxonomy across four prompting strategies. Panel A: distribution of error types among failed runs. Panel B: location of the first error in the generated file.

Panel A: Error Type Distribution

Error Type	Method	n	%
Parse: Bad Module Body	FS	228	47.8
	HC	264	46.3
	Prog.	243	49.8
	FIM	290	52.0
Parse: Other	FS	72	15.1
	HC	227	39.8
	Prog.	70	14.3
	FIM	191	34.2
Parse: Unexpected Token	FS	82	17.2
	HC	29	5.1
	Prog.	65	13.3
	FIM	17	3.0

Panel B: First Error Location

Location	Method	n	%
Early (first third)	FS	240	50.3
	HC	63	11.1
	Prog.	220	45.1
	FIM	81	14.5
Mid (middle third)	FS	196	41.1
	HC	261	45.8
	Prog.	209	42.8
	FIM	310	55.6
Late (final third)	FS	41	8.6
	HC	246	43.2
	Prog.	59	12.1
	FIM	167	29.9

Table 5: Per-model SANY (S) and TLC (T) pass counts across all four prompting strategies (25 open-weight models, 26 specifications each). Detailed per-model per-specification breakdowns are provided in supplementary materials.

		FS		HC		Prog.		FIM	
Family	Model	S	T	S	T	S	T	S	T
DeepSeek	r1:70b	0	0	3	0	3	0	3	0
	r1:32b	6	0	2	0	8	1	6	0
	r1:14b	1	0	2	0	2	1	2	0
	r1:8b	22	0	1	0	15	14	3	0
	r1:7b	1	0	4	0	6	1	3	0
	r1:1.5b	1	0	2	0	8	3	2	0
	coder:6.7b	1	0	1	0	0	0	2	0
	coder:latest	0	0	5	0	3	1	5	0
LLaMA	3.3:70b	11	0	2	0	18	0	6	0
	3.2:latest	11	0	1	0	18	0	5	0
	3.1:8b	6	0	3	0	3	0	4	0
	3:latest	3	0	3	0	2	0	5	0
	3.2:1b	2	0	2	0	12	0	4	0
Qwen	3:235b	22	0	3	0	21	12	5	0
	3:30b	23	0	3	0	11	10	5	0
	3:4b	5	0	3	0	1	1	2	0
QwQ	latest	3	0	9	0	2	0	10	0
Phi	4-mini:latest	5	0	4	0	4	4	1	0
Granite	3.3:latest	3	0	5	0	2	0	2	0
GPT-OSS	120b	17	0	4	0	6	1	3	0
	20b	18	0	3	0	9	6	5	0
CodeLLaMA	latest	3	0	5	0	2	0	3	0
Mistral	latest	3	0	2	0	3	0	3	0
Gemma	2b	6	0	4	0	2	1	2	0
Starling-LM	latest	0	0	2	0	1	0	1	0
Average		**6.920.00**		**3.170.00**		**6.382.08**		**3.71**	**0.00**

FS: Few Shot, **HC**: Half Completion, **Prog.**: Progressive, **FIM**: Fill-in-Middle

METHOD

Dataset

- We use the TLA+ Examples repository maintained by the TLA+ Foundation, the primary source of community-written formal specifications
- The repository contains 205 TLA+ specifications (at the time of writing) across 98 projects:
  - distributed consensus protocols (Paxos, Raft, Byzantine agreement)
  - concurrency problems (dining philosophers, readers-writers)
  - algorithmic puzzles(Towers of Hanoi, N-queens)
- We test across 30 LLMs from eight model families
- Test four prompting strategies adapted from common LLM code generation settings
  - Few-Shot Prompting: in context learning with k=3 examples from training set
  - Progressing Prompting: Also uses k=3 few-shot examples but adds instructional messages that guide the model through the specification structure
  - Fill-in-Middle Prompting: Code completion strategy splits the ground-truth specification into three parts prefix (30%), middle (40%), and suffix (30%)
  - Half Completion Prompting: Provides the LLM the first 50% of the ground-truth as a prefix, along with comments, and configuration. The LLM is prompted to generate the remaining 50%
- For completion strategy prompting we compute the BLEU, ROUGE-L, edit distance, exact match, and line accuracy against the ground-truth segment

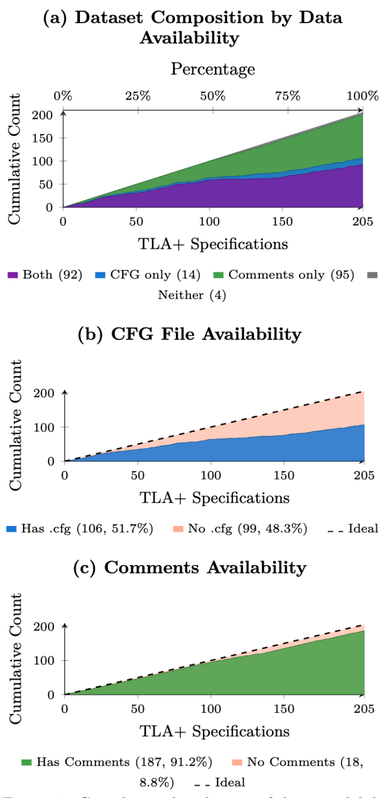


Figure 1: Cumulative distribution of data availability across 205 TLA+ specifications. (a) Four-category breakdown. (b) CFG presence: 51.7% include configuration files. (c) Comments: 91.2% contain natural language descriptions.

2.1.1 Few-Shot Prompting

System Message

```
You are a TLA+ code generator.
You must output ONLY valid TLA+ code that
can be parsed by the TLA+ Toolbox.
You must NEVER include markdown fences ("")
or explanations.
You must NEVER include reasoning text,
English descriptions, or comments.
You must NEVER output placeholders like
<ModuleName> or <constant> literally.
Strict formatting rules:
```

- Output ONLY the complete TLA+ module.
- The module must start with exactly this line: `-- MODULE ModuleName --`
- The module must end with exactly this line: `====`
- Include only valid TLA+ constructs: EXTENDS, CONSTANTS, VARIABLES, Init, Next, Spec.
- Always define every symbol before using it.
- After the module, include a TLC configuration section.

User Message Structure

```
Each few-shot example HumanMessage contains:
# Comments:{comments}
optionally followed by:
# TLC Configuration:{cfg}
Each example AIMessage contains:
-- MODULE {module_name} --
{full_tla}
====
For the target inference, the final message is:
# Comments:{target_comments}
plus optional TLC configuration.
```

2.1.2 Progressive Prompting

System Message

```
You are a TLA+ code generator. Output
ONLY valid TLA+ code. NO markdown fences,
explanations, reasoning, or placeholders.
```

Instruction Message 1: Basic Structure

```
Start: -- MODULE ModuleName --
End: ====
Required: EXTENDS, CONSTANTS, VARIABLES,
Init, Next, Spec
```

Instruction Message 2: Syntax Reminders

```
[A]_v is temporal formula: A / v'=v
Inequality: use # or /=
Init: initial state predicate
Next: next-state relation
Spec: Init /\ [][Next]_<vars>
```

Instruction Message 3: Conventions

```
TypeOK for type invariants
Define all symbols before use
Include TLC config after ====
Format:
TLC Configuration:
CONSTANTS <constant> = <value>
SPECIFICATION Spec
INVARIANTS <names>
```

Final Instruction

```
Generate complete TLA+ module with TLC
config. Follow all previous rules and
syntax.
```

2.1.3 Fill-in-Middle (FIM)

System Message

```
You are a TLA+ code completion assistant
using fill-in-the-middle approach.
You will be given:
```

- Comments describing the TLA+ specification
- Optional TLC configuration
- PREFIX: The beginning portion of the TLA+ code
- SUFFIX: The ending portion of the TLA+ code

```
Your task is to generate ONLY the MIDDLE
section that connects the PREFIX and
SUFFIX.
CRITICAL INSTRUCTIONS:
```

- Output ONLY the middle section code that fills the gap between PREFIX and SUFFIX
- DO NOT repeat the PREFIX code in your output
- DO NOT repeat the SUFFIX code in your output
- Your output should start immediately after where PREFIX ends
- Your output should end immediately before where SUFFIX begins
- NEVER include markdown fences or explanations

User Message Template

```
# Comments:{target_comments}
# TLC Configuration:{target_cfg} (optional)
# PREFIX (first 30% of code):{prefix}
# SUFFIX (last 30% of code):{suffix}
# YOUR TASK: Generate ONLY the MIDDLE
section that connects PREFIX and SUFFIX.
# DO NOT include PREFIX or SUFFIX in your
output!
```

2.1.4 Half Completion

System Message

```
You are a TLA+ code completion assistant.
You will be given:
```